

GPU 101

How a machine for pictures learned to compute $A + B$

Minjae Gwon

POSTECH

September 10, 2026

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

3. The execution engine

- Warps and SIMT
- SMs and GPU organization

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

3. The execution engine

4. Following A + B

What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

Pictures → Execution engines → $C = A + B$

One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

CPU vs. GPU

One paintball at a time.

Many paintballs together.

► [Watch the demo](#)

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

1. Motivation

2. From pictures to programs

- Rendering and shaders
 - GPGPU and CUDA
-

3. The execution engine

4. Following A + B

The original deadline: draw the next frame

3D rendering

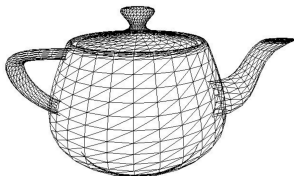


Image credit: Henrik Wann Jensen

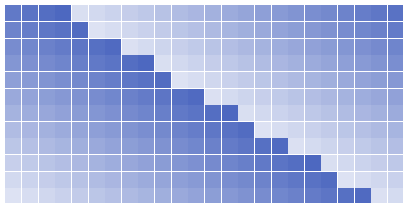
Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Geometry + materials + lights + camera → an image

Millions of positions. The same kind of calculation.



Different positions, different inputs

$$1920 \times 1080$$

output pixels per frame

$$\times 60 \text{ frames / second}$$

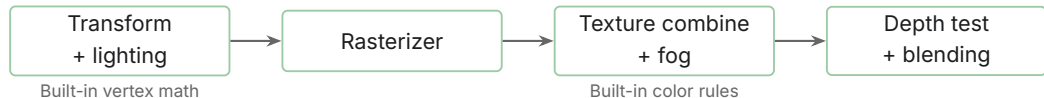
$$\approx 124 \text{ million}$$

output pixels per second

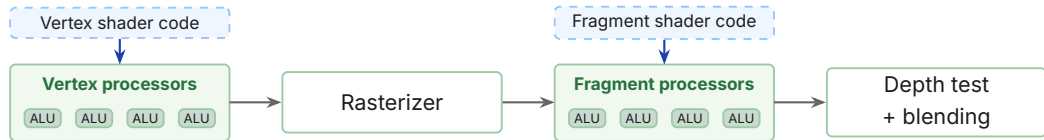
The opportunity is repeated work across independent data.

Richer materials needed programmable hardware

EARLY OPENGL · SETTINGS FOR FIXED-FUNCTION HARDWARE



EARLY PROGRAMMABLE GPU · APPLICATION SUPPLIES PROGRAMS



Pipeline order, not a floorplan · texture sampling hardware remains · output goes to the framebuffer

Custom material formulas need both code and processors that can execute it.

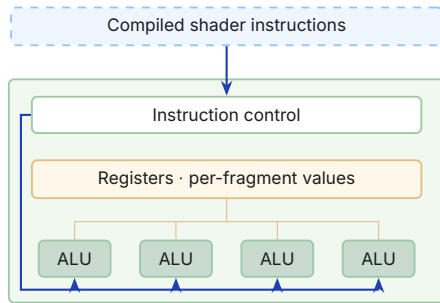
NVIDIA · The Cg Tutorial, Chapter 1 · simplified hardware stages and illustrative unit counts

Shader code becomes instructions for physical processors

SOFTWARE · ONE FRAGMENT'S WATER SHADING

```
normal = ripple_normal(uv, time)
reflected = sample_reflection(view, normal)
weight = reflection_weight(view, normal)
output = mix(water_color, reflected, weight)
```

- The program chooses the calculation.
- Each fragment supplies different inputs.



The same units process many fragments.

Code supplies instructions; registers hold values; ALUs perform the arithmetic.

Do those numbers have to be colors?

THE GRAPHICS INTERPRETATION



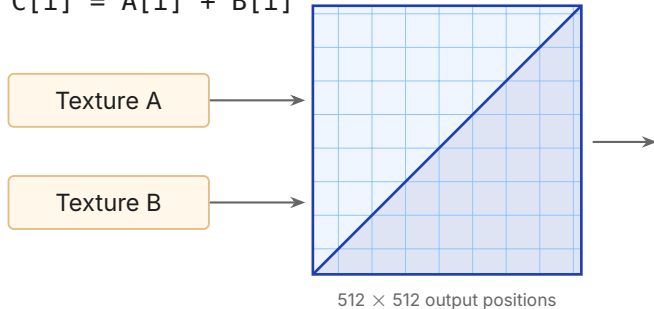
THE COMPUTE INTERPRETATION



GPGPU: General-Purpose Computing on GPUs

To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Interpret the output as an array.

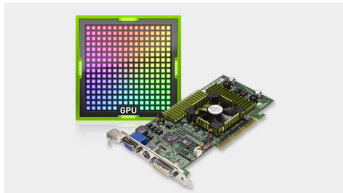
A draw call to the GPU; an array operation to the researcher.

“All this just to do $A + B$ ”



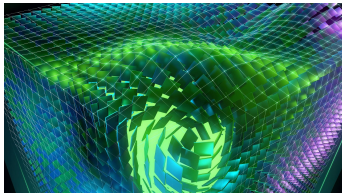
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

CUDA arrived with a more general GPU



1999

GeForce 256



2006

CUDA architecture

`launch()`

CUDA Toolkit 1.0

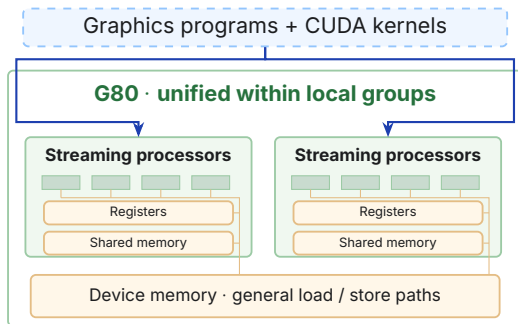
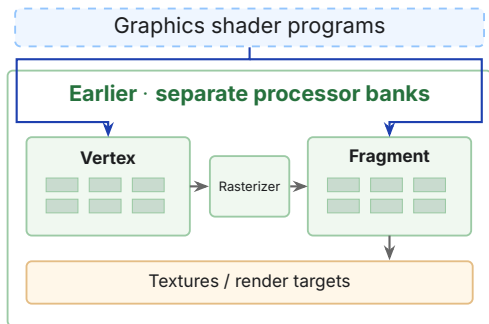
2007

CUDA Toolkit 1.0

A compute programming model arrived with a GPU that could execute it.

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

CUDA used unified processors and new memory paths



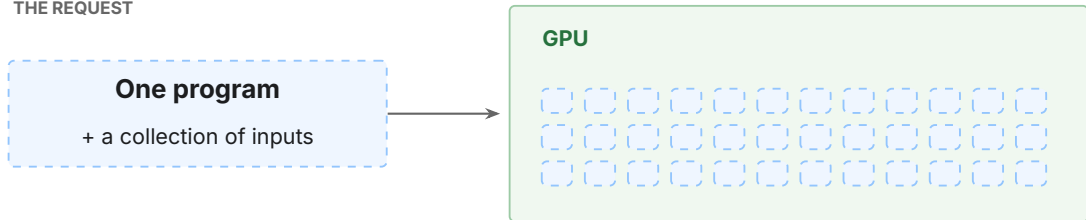
Selected resources · graphics rasterization remains · processor counts are illustrative

CUDA needed executable instructions, direct data access, and local cooperation.

NVIDIA · GeForce 8800 Technical Brief, pp. 12–14, 21–25 · CUDA Guide 1.0, pp. 2–5

“Run this program many times.”

THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

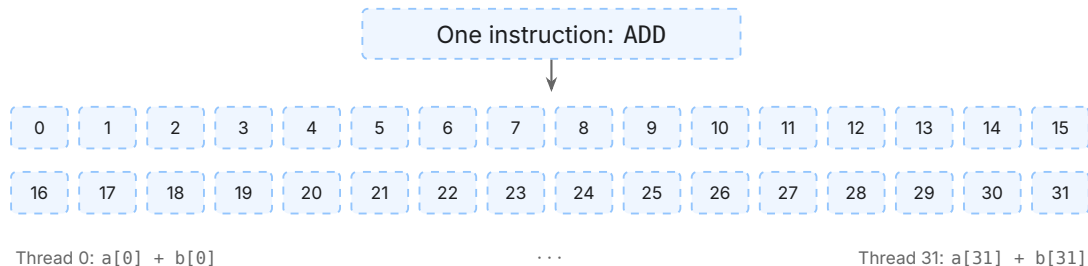
2. From pictures to programs

3. The execution engine

- Warps and SIMT
 - SMs and GPU organization
-

4. Following A + B

Hardware groups 32 logical threads into a warp

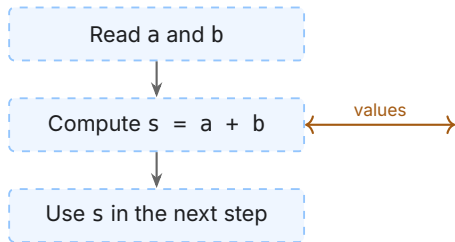


SIMT Single Instruction, Multiple Threads

A warp groups thread execution; execution units do the work.

Each thread needs values that survive an instruction

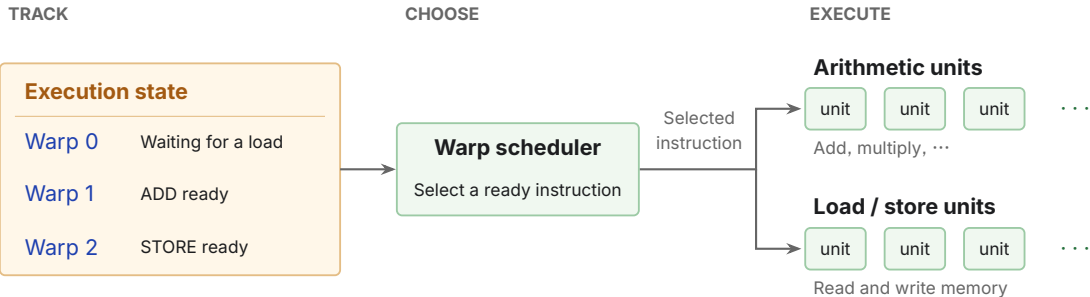
ONE WARP · SAME STEPS, DIFFERENT VALUES



REGISTER FILE			
Thread	a	b	s
0	a[0]	b[0]	s[0]
1	a[1]	b[1]	s[1]
	...		
31	a[31]	b[31]	s[31]

A register file holds working values; each thread has its own registers.

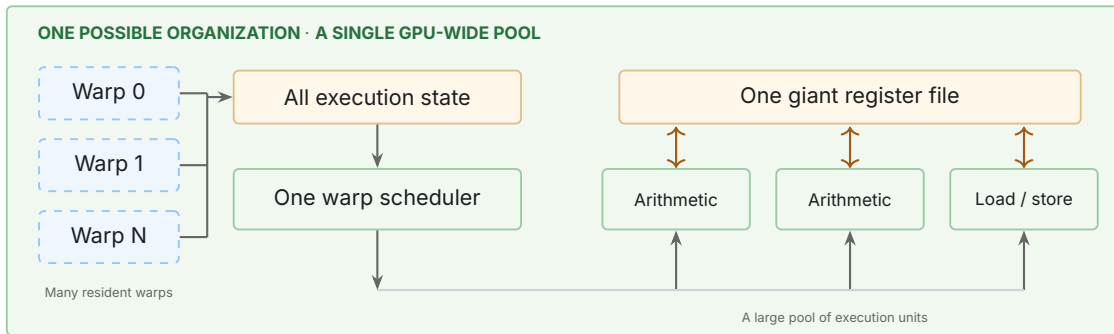
Track progress, choose an instruction, execute it



Required roles; unit counts and connections are illustrative.

Track each warp's state. Issue selected instructions to execution units.

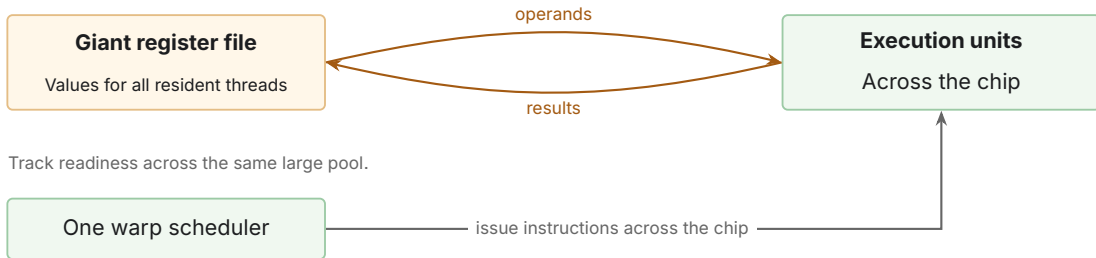
Why not build one giant execution engine?



One place for values. One place to choose work. Many units to execute it.

The values and control signals must travel too

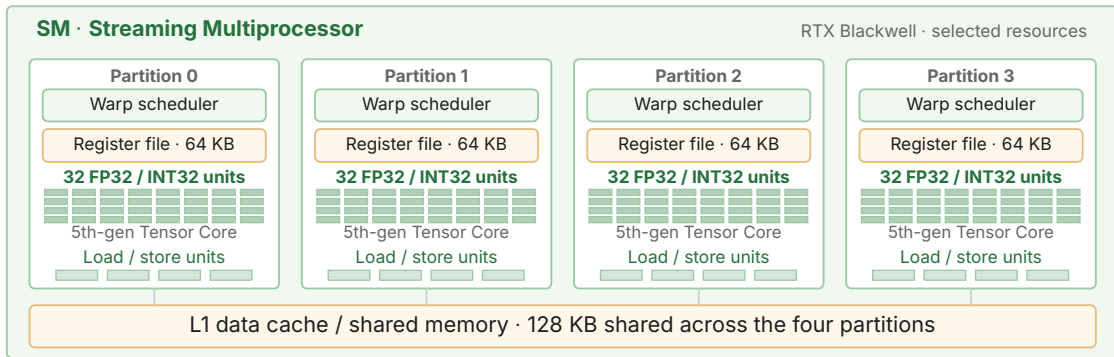
THE SAME CENTRALIZED DESIGN · NOW CONSIDER THE CONNECTIONS



Track readiness across the same large pool.

More units also need more register bandwidth and a larger control network.

An SM keeps execution and its working state close

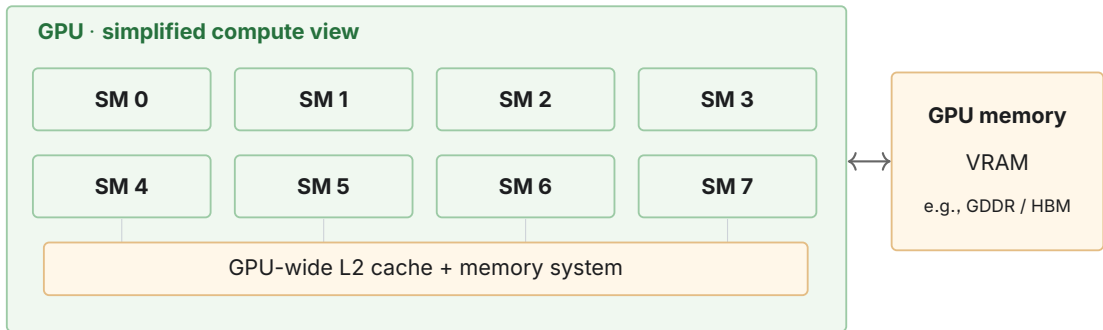


Each scheduler manages many warps using its partition's local resources.

NVIDIA RTX Blackwell whitepaper · Figure 5, pp. 10–12 · simplified and rearranged

The execution engine

Scale the GPU by repeating that execution engine



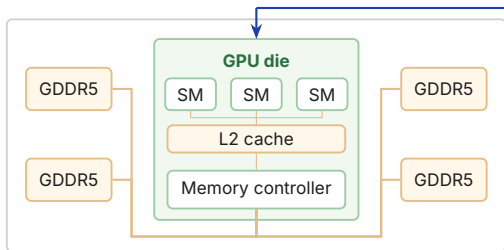
Share control within a warp; keep execution resources local to an SM.

NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

More memory traffic changed the GPU package

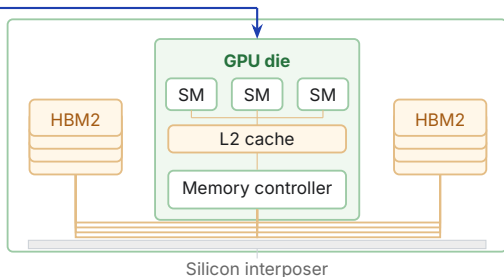
CUDA kernel · load inputs → compute → store outputs

GDDR5 · SEPARATE PACKAGES ON A BOARD



Same load / store model · schematic components and wiring

P100 · 2016 · GPU AND HBM2 IN ONE PACKAGE



HBM widens the physical data path; kernels still need efficient access and reuse.

NVIDIA · Inside Pascal (2016) · Dramatic Improvements in Memory · schematic packaging comparison

Matrix kernels gained a new physical execution path

MATRIX MULTIPLY + ACCUMULATE · $D = A \times B + C$

GEMM kernel · repeated multiply-adds

FMA instructions: $d \leftarrow a \times b + d$

Pascal SM · selected resources

Registers · operands and accumulated values

ALU

ALU

ALU

ALU

ALU

ALU

CUDA 9 · library or WMMA kernel

Matrix multiply-accumulate instructions

Volta SM · selected resources

Registers · operands and accumulated values

ALU

ALU

ALU

Tensor
Core

Tensor
Core

Ordinary ALUs remain

Example: FP16 matrix inputs, FP32 accumulation · illustrative unit counts · instruction throughput is not shown

Tensor-aware kernels and instructions use added matrix hardware.

NVIDIA · Programming Tensor Cores in CUDA 9 (2017) · Pascal FMA and Volta Tensor Core paths

Generating text adds a growing state to reuse



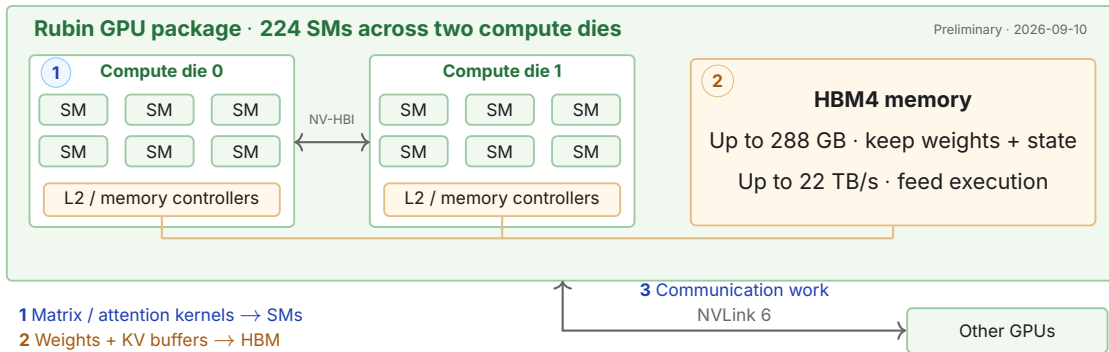
Longer context + more requests → more stored state

Generating tokens → repeated access to weights and state

A model spread across GPUs → communication

The bottleneck depends on the model, batch size, and execution phase.

Rubin scales compute, memory, and communication



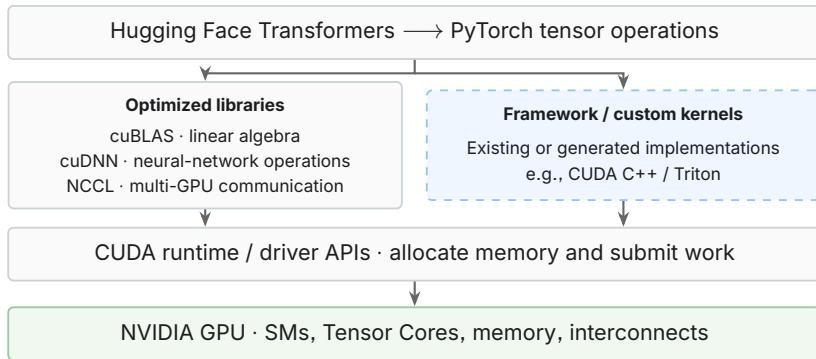
The workload shapes the arithmetic, storage, and connections together.

NVIDIA · Inside Rubin GPU Architecture · July 2026 · SM counts, wiring, and software mappings are schematic

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

CUDA connects frameworks to the hardware



Build

Compilers
e.g., nvcc

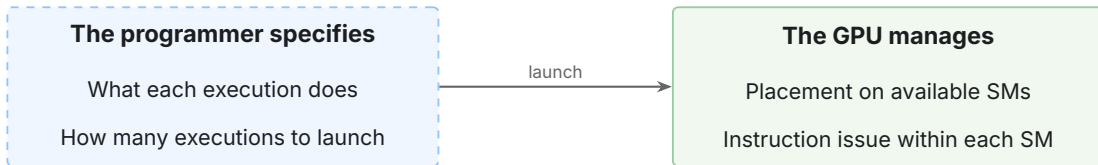
Measure

Nsight Systems
Application timeline
Nsight Compute
Individual kernels

Libraries, kernels, compilers, and tools make hardware capabilities usable.

NVIDIA CUDA platform and libraries · PyTorch / Hugging Face docs · example execution paths

Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

A work description is not a physical placement plan.

Start with one line of PyTorch

```
import torch  
A = torch.randn(1024, 1024, device="cuda")  
B = torch.randn(1024, 1024, device="cuda")  
C = A + B
```

A



+

B



=

C



Each output needs only its own two inputs. No neighbor must finish first.

If the input starts in CPU memory, copy it to the GPU

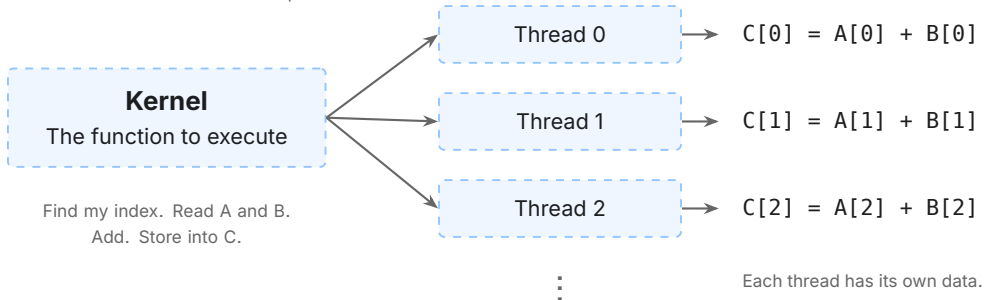


```
A_cpu = torch.randn(1024, 1024, device="cpu")  
A = A_cpu.to("cuda")
```

With `device="cuda"`, our original example already creates GPU tensors.

One program, many logical executions

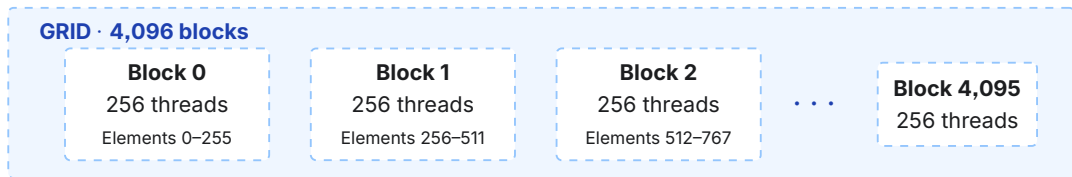
Educational kernel · one element per thread · matrix elements numbered as a flat buffer



A kernel is code. A thread is one logical execution of that code.

How many executions do we launch?

$1024 \times 1024 = 1,048,576$ elements



```
// CUDA C++: A/B/C buffers are ready; device_a/b/c are their GPU addresses.  
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

4,096 blocks $\times$ 256 threads = 1,048,576 logical threads.

Every thread can find its own element

```
__global__ void add(  
    const float* a,  
    const float* b,  
    float* c,  
    int element_count  
) {  
    int index = blockIdx.x * blockDim.x  
        + threadIdx.x;  
    if (index < element_count) {  
        c[index] = a[index] + b[index];  
    }  
}
```

Same code, different built-in IDs, different data.

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

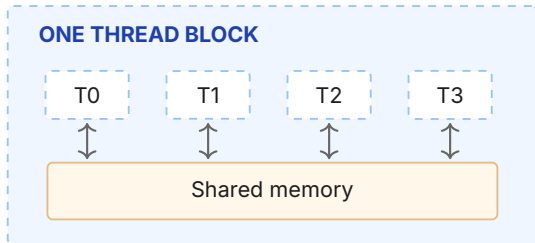
778

$C[778] = A[778] + B[778]$

The bounds check protects the final block when the size is not divisible by 256.

__global__: GPU kernel .x: 1D launch IDs

A block is also a cooperation boundary



Share data locally

Threads in a block can exchange data.

Synchronize within the block

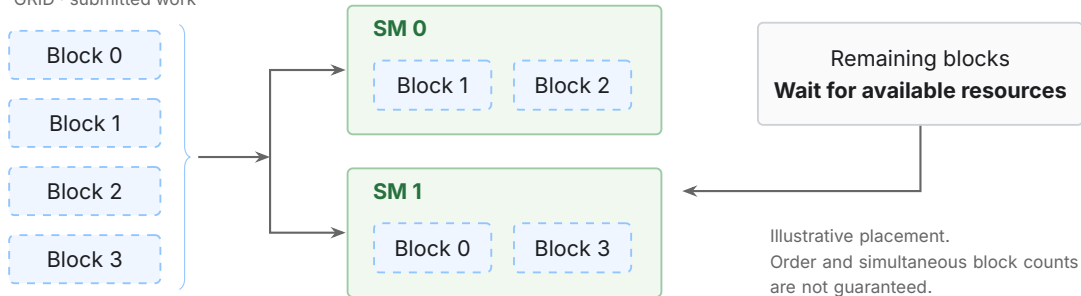
`__syncthreads()` provides a block-wide barrier.

For example, threads can combine intermediate results through shared memory.

This addition needs no shared intermediate values or block barrier.

Many blocks share a finite set of SMs

GRID · submitted work



One block runs on one SM. An SM may host several blocks.

A 256-thread block contains eight warps

BLOCK 0 · 256 threads

Warp 0  0–31

Warp 1  32–63

Warp 2  64–95

Warp 3  96–127

Warp 4  128–159

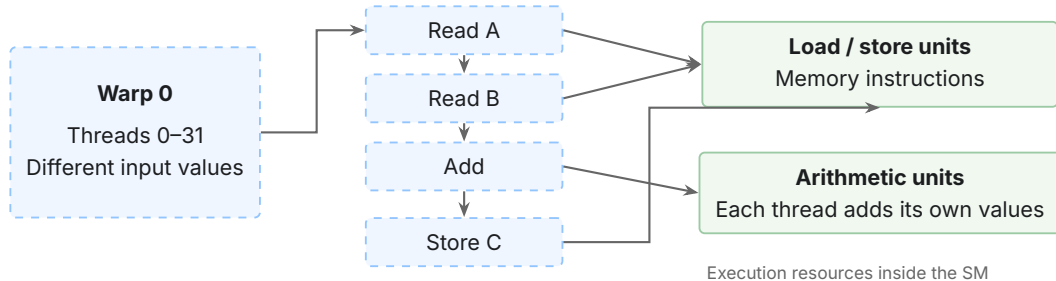
Warp 5  160–191

Warp 6  192–223

Warp 7  224–255

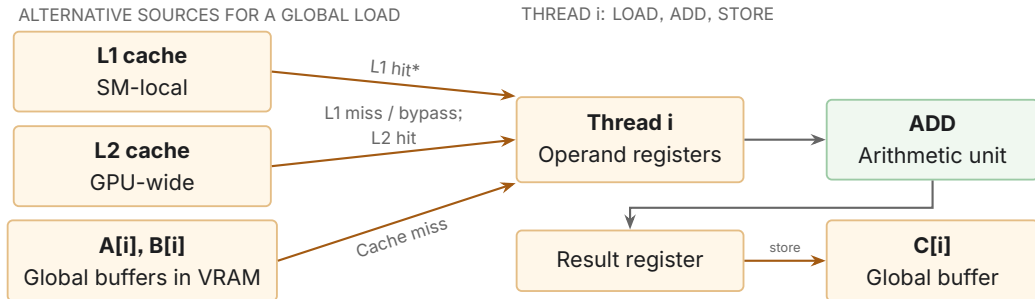
Within a block, consecutive groups of 32 threads form warps.

A warp needs different units for different instructions



The instruction is shared; the data belongs to each thread.

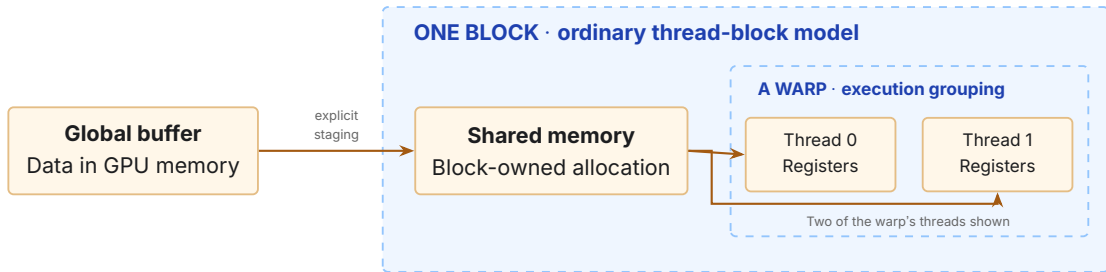
For this addition, operands are loaded into thread registers



*L1 use depends on architecture and policy. Arrows show possible data sources, not physical routes.

The add kernel uses per-thread registers. Shared-memory staging is unnecessary.

Shared memory is explicit storage for block cooperation

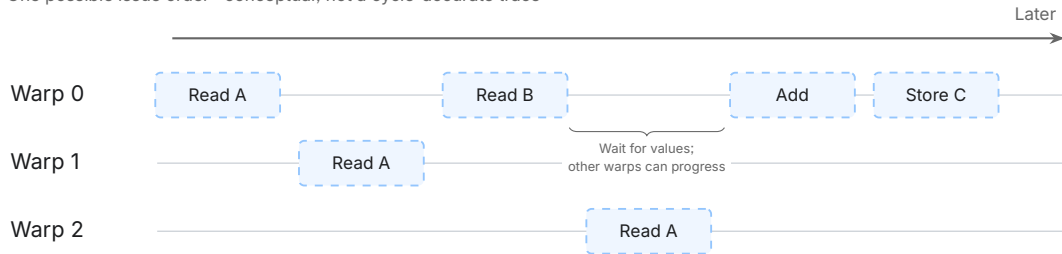


Useful when threads reuse or exchange data. This addition does not take this optional path.

Shared memory is optional staging that the kernel manages for cooperation and reuse.

The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



Registers retain each thread's values between instructions.

A warp does not have to finish its whole program before another runs.

Many resident warps share the same execution units

RTX BLACKWELL · COMPUTE CAPABILITY 12.0

Resident execution state

Up to 48 warps per SM

$48 \times 32 = 1,536$ logical threads

Limited by registers, shared memory, and block size



Physical arithmetic resources

4 partitions $\times$ 32 FP32 / INT32 units

= 128 units per SM

Schedulers issue ready instructions to these units

Resident threads retain their state; they do not each own an arithmetic unit.
Warp width, instruction throughput, and result latency are separate quantities.

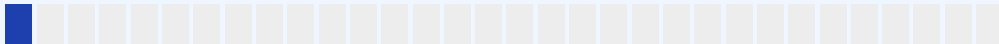
Resident warp capacity is not the arithmetic-unit count divided by 32.

What if we launch just one thread?

```
add<<<1, 1>>>(device_a, device_b, device_c, 1);
```

ONE SM

BLOCK 0 · WARP 0



Lane 0: one valid thread

Lanes 1–31: unused

One active thread in a warp on one SM.

More threads in one block still means one SM

ONE LARGE BLOCK

`add<<<1, 1024>>>(...)`

Small example · 1,024 elements

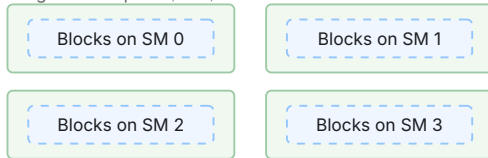


1,024 threads/block: device and kernel limits apply

MANY BLOCKS

`add<<<4096, 256>>>(...)`

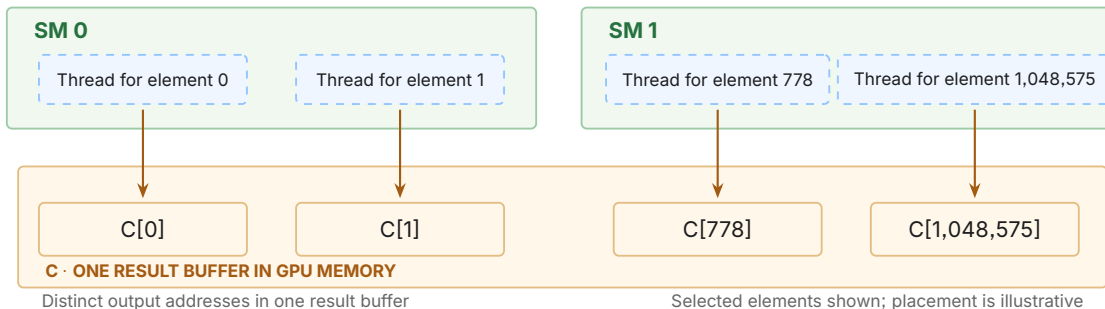
Original example · 1,048,576 elements



Blocks available for multiple SMs

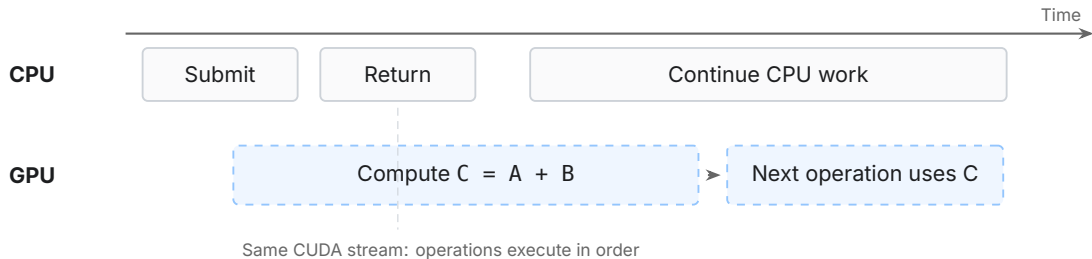
Fill the warps, and provide blocks that can spread across SMs.

Threads write directly into C



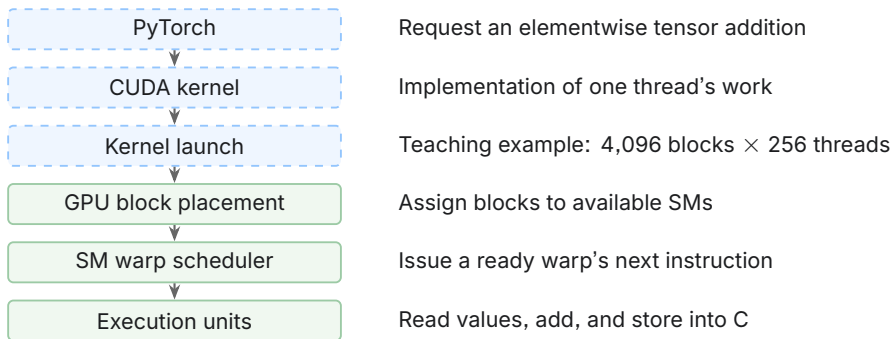
Each thread stores its own result. The CPU does not reassemble the matrix.

Python can return before the GPU finishes



- GPU operations are normally submitted asynchronously.
- Ordinary CPU copies wait as needed; GPU timing must account for completion.

Follow the request all the way down



One tensor operation becomes many scheduled instructions on real hardware.

Keep the work and the machine separate

Kernel	The GPU program to execute
Thread	One logical execution of that program
Block	A group of threads that can cooperate
Grid	All blocks submitted for this kernel launch
Warp	A 32-lane execution group within one block

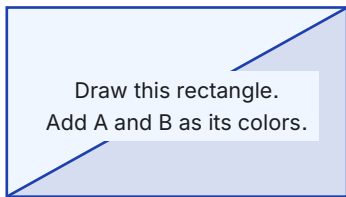
SM	The hardware engine that executes resident warps
-----------	--

Grid $\neq$ GPU. Block $\neq$ SM. Thread $\neq$ a dedicated core.

Following $A + B$

From two triangles to one line

THEN



NOW



$$C = A + B$$

The constant idea: apply the same computation to many inputs.

GPU 101

How a machine for pictures learned to compute $A + B$

Educational scope This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.